EXPERIMENT 9: Multilayer Perceptron Implementation

1. Aim

To build, train and evaluate a Multilayer Perceptron (MLP) classifier using TensorFlow/Keras to predict customer churn, preprocess numerical and categorical features, evaluate the model using classification metrics, and study the effect of hidden-layer width.

2. Mathematical Formulae

Forward propagation

$$Z^{[l]} = A^{[l-1]}W^{[l]} + b^{[l]}$$

$$A^{[l]} = ReLU(Z^{[l]})$$

ReLU activation

$$ReLU(z) = \max(0, z)$$

Sigmoid output

$$\hat{y} = \frac{1}{1 + e^{-z}}$$

Binary cross-entropy

$$L = -\frac{1}{m}\sum_{i=1}^{m}[y_i \log(\hat{y}_i) + (1 - y_i)\log(1 - \hat{y}_i)]$$

Accuracy

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

Precision and Recall

$$Precision = \frac{TP}{TP + FP}$$

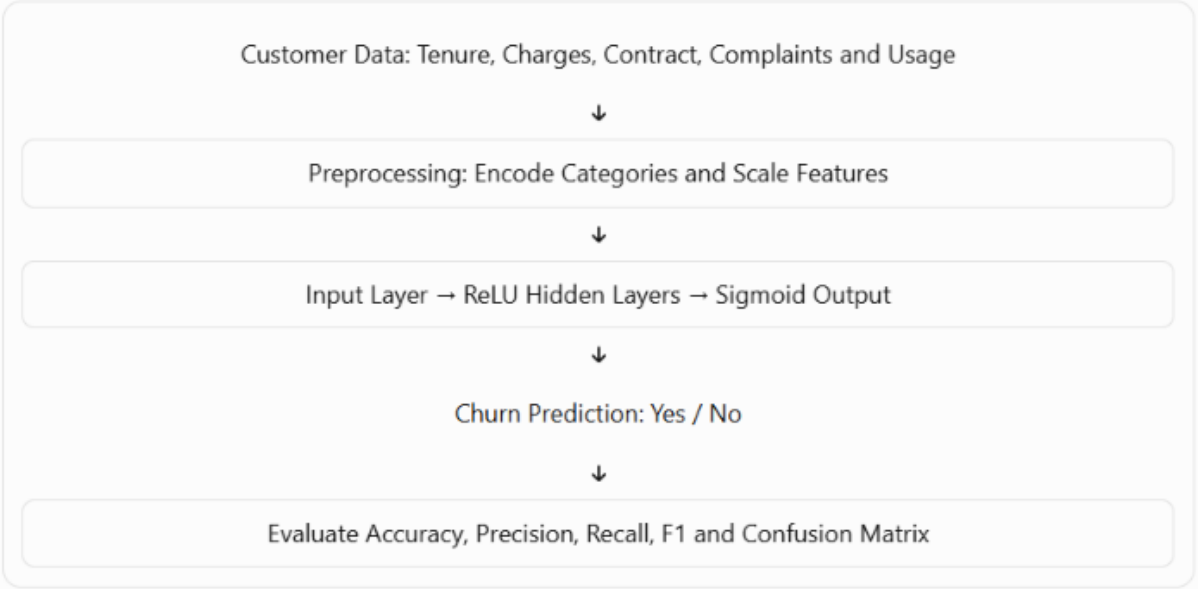
$$Recall = \frac{TP}{TP + FN}$$

F1-score

$$F1 = 2\frac{Precision \times Recall}{Precision + Recall}$$

Where TP, TN, FP and FN represent true positives, true negatives, false positives and false negatives.

3. System Architecture



4. Algorithm / Procedure

1. Generate a synthetic customer-churn dataset.
2. Encode categorical features and standardize numerical features.
3. Split the data into training, validation and test sets.
4. Build an MLP using two hidden layers with ReLU activation and a sigmoid output.
5. Compile the model using Adam and binary cross-entropy.
6. Train the model and plot training and validation curves.
7. Evaluate the test set using accuracy, precision, recall, F1-score and a confusion matrix.
8. Repeat training with different hidden-layer widths and compare results.

5. Python Program

```
import numpy as np

import pandas as pd

import matplotlib.pyplot as plt

import tensorflow as tf


from sklearn.model_selection import train_test_split

from sklearn.preprocessing import StandardScaler

from sklearn.metrics import (
```

```
    accuracy_score, precision_score, recall_score,
    f1_score, confusion_matrix, ConfusionMatrixDisplay
)

np.random.seed(42)
tf.random.set_seed(42)

# Generate synthetic customer data
n = 1200

tenure = np.random.randint(1, 73, n)
monthly_charges = np.random.uniform(20, 120, n)
data_usage = np.random.uniform(1, 100, n)
complaints = np.random.poisson(1.0, n)

contract = np.random.choice(
    ["Monthly", "OneYear", "TwoYear"],
    size=n,
    p=[0.6, 0.25, 0.15]
)

# Create a synthetic churn probability
logit = (
    1.2
    - 0.035 * tenure
    + 0.018 * (monthly_charges - 60)
    + 0.45 * complaints
    + 0.7 * (contract == "Monthly")
    - 0.5 * (contract == "TwoYear")
)
```

```
probability = 1 / (1 + np.exp(-logit))
churn = np.random.binomial(1, probability)

df = pd.DataFrame({
    "tenure": tenure,
    "monthly_charges": monthly_charges,
    "data_usage": data_usage,
    "complaints": complaints,
    "contract": contract,
    "Churn": churn
})

# Encode categorical feature
X = pd.get_dummies(
    df.drop(columns="Churn"),
    columns=["contract"],
    dtype=int
)

y = df["Churn"].to_numpy()

# Split data
X_train, X_temp, y_train, y_temp = train_test_split(
    X, y, test_size=0.30, random_state=42, stratify=y
)

X_val, X_test, y_val, y_test = train_test_split(
    X_temp, y_temp, test_size=0.50,
    random_state=42, stratify=y_temp
)
```

```
# Standardize numerical features only
numeric_columns = [
    "tenure", "monthly_charges", "data_usage", "complaints"
]

scaler = StandardScaler()

X_train = X_train.copy()
X_val = X_val.copy()
X_test = X_test.copy()

X_train.loc[:, numeric_columns] = scaler.fit_transform(
    X_train[numeric_columns]
)

X_val.loc[:, numeric_columns] = scaler.transform(
    X_val[numeric_columns]
)

X_test.loc[:, numeric_columns] = scaler.transform(
    X_test[numeric_columns]
)

X_train = X_train.to_numpy(dtype=np.float32)
X_val = X_val.to_numpy(dtype=np.float32)
X_test = X_test.to_numpy(dtype=np.float32)

def build_model(width):
    model = tf.keras.Sequential([
        tf.keras.layers.Input(shape=(X_train.shape[1],)),
        tf.keras.layers.Dense(width, activation="relu"),
```

```
        tf.keras.layers.Dense(width // 2, activation="relu"),
        tf.keras.layers.Dense(1, activation="sigmoid")
    ])

    model.compile(
        optimizer="adam",
        loss="binary_crossentropy",
        metrics=["accuracy"]
    )

    return model

# Compare hidden-layer widths
results = []

for width in [16, 32, 64]:
    tf.keras.backend.clear_session()
    tf.random.set_seed(42)

    model = build_model(width)

    history = model.fit(
        X_train, y_train,
        validation_data=(X_val, y_val),
        epochs=30,
        batch_size=32,
        verbose=0
    )

    probabilities = model.predict(X_test, verbose=0).ravel()
    predictions = (probabilities >= 0.5).astype(int)
```

```
    results.append({
        "Width": width,
        "Accuracy": accuracy_score(y_test, predictions),
        "Precision": precision_score(
            y_test, predictions, zero_division=0
        ),
        "Recall": recall_score(
            y_test, predictions, zero_division=0
        ),
        "F1": f1_score(
            y_test, predictions, zero_division=0
        )
    })

    # Plot curves for this model
    plt.figure(figsize=(7, 4))
    plt.plot(history.history["loss"], label="Training Loss")
    plt.plot(history.history["val_loss"], label="Validation Loss")
    plt.xlabel("Epoch")
    plt.ylabel("Binary Cross-Entropy Loss")
    plt.title(f"MLP Loss Curves: Width {width}")
    plt.legend()
    plt.show()

    if width == 32:
        final_model = model
        final_predictions = predictions

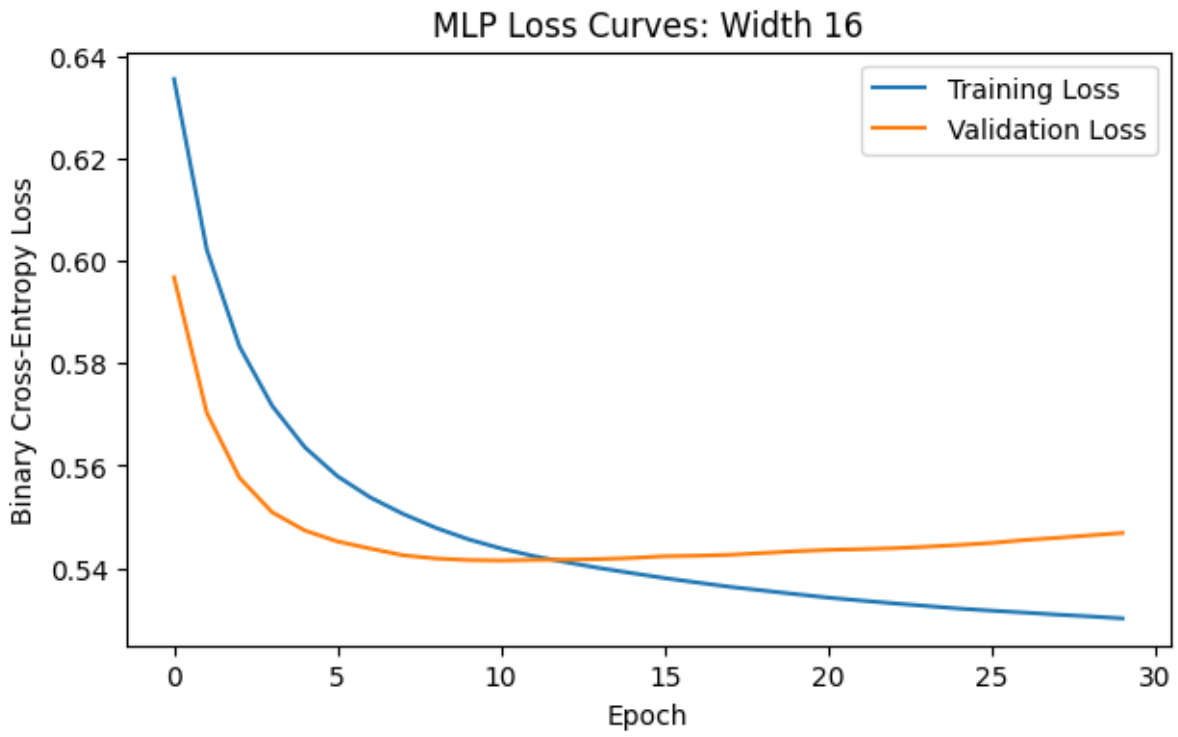
print("\nModel Comparison")
print(pd.DataFrame(results).round(4))
```

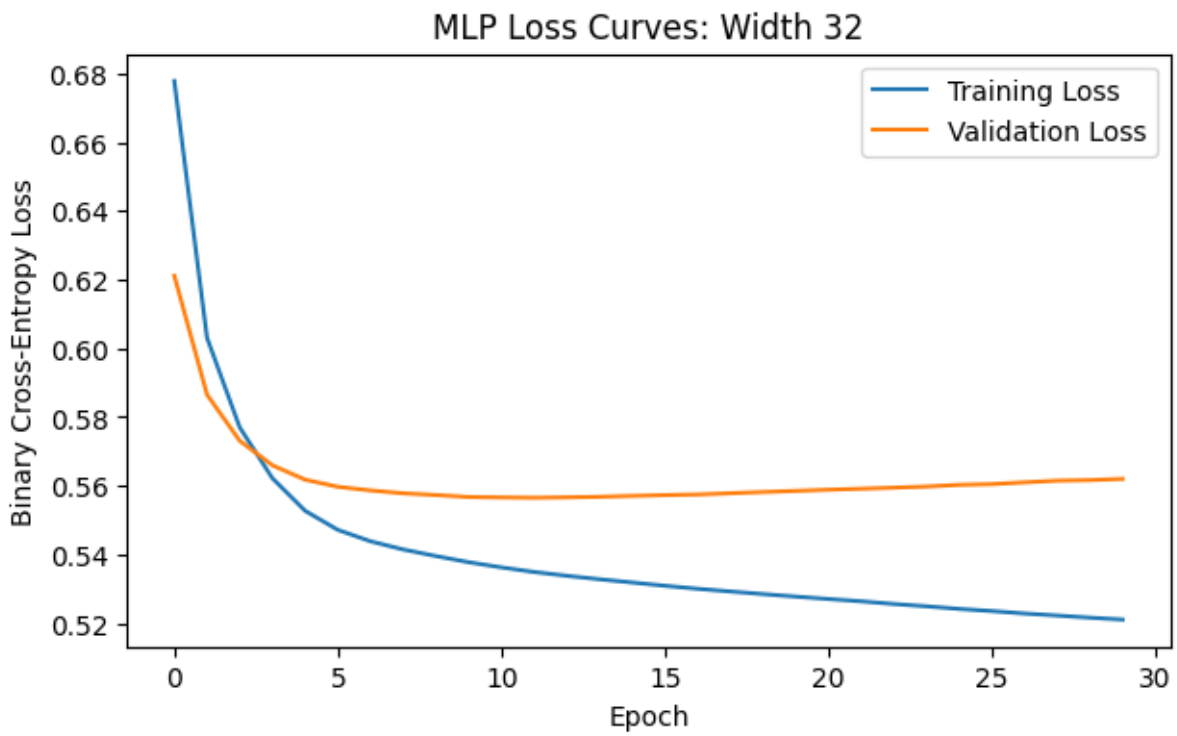
```
# Confusion matrix for width 32
cm = confusion_matrix(y_test, final_predictions)

display = ConfusionMatrixDisplay(
   confusion_matrix=cm,
   display_labels=["No Churn", "Churn"]
)

display.plot()
plt.title("Customer Churn Confusion Matrix")
plt.show()
```

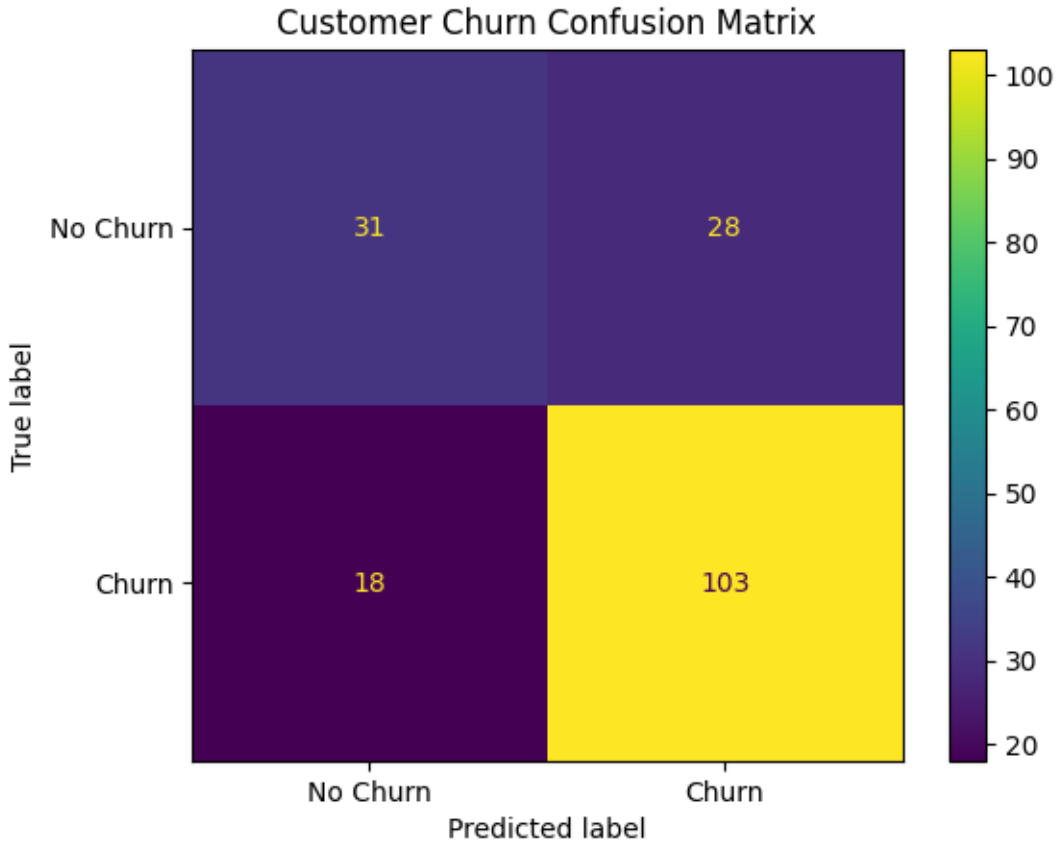
6. Output





Model Comparison

	Width	Accuracy	Precision	Recall	F1
0	16	0.7389	0.7761	0.8595	0.8157
1	32	0.7444	0.7863	0.8512	0.8175
2	64	0.7111	0.7760	0.8017	0.7886



7. Result

A Multilayer Perceptron was developed using TensorFlow/Keras to classify customer churn. Numerical and categorical data were preprocessed, the model was trained using Adam and binary cross-entropy, and its performance was evaluated using classification metrics and a confusion matrix. Different hidden-layer widths were compared.